

EC-241 Object-Oriented Programming

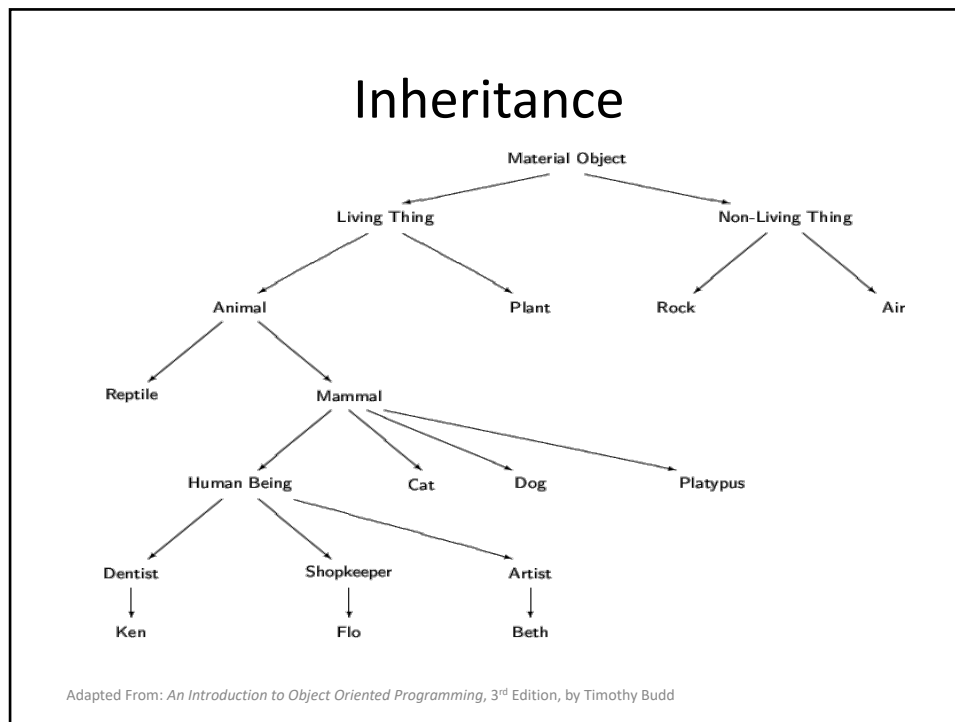
LECTURE 8

1

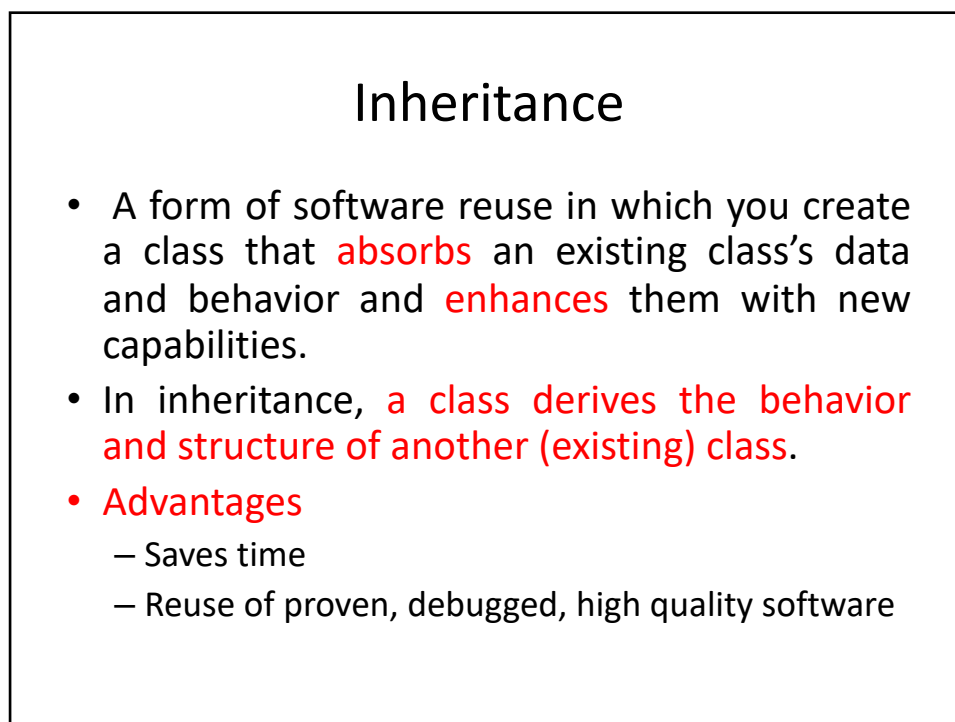
General Info

- Object Oriented Programming provides:
 - Encapsulation
 - Inheritance
- That improves design, structure and reusability of code.

2

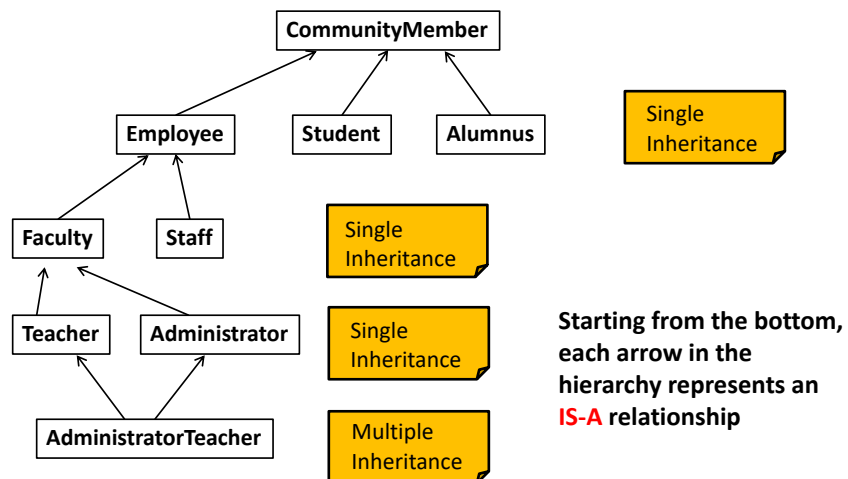


3



4

Example: Inheritance Hierarchy for University Community Member

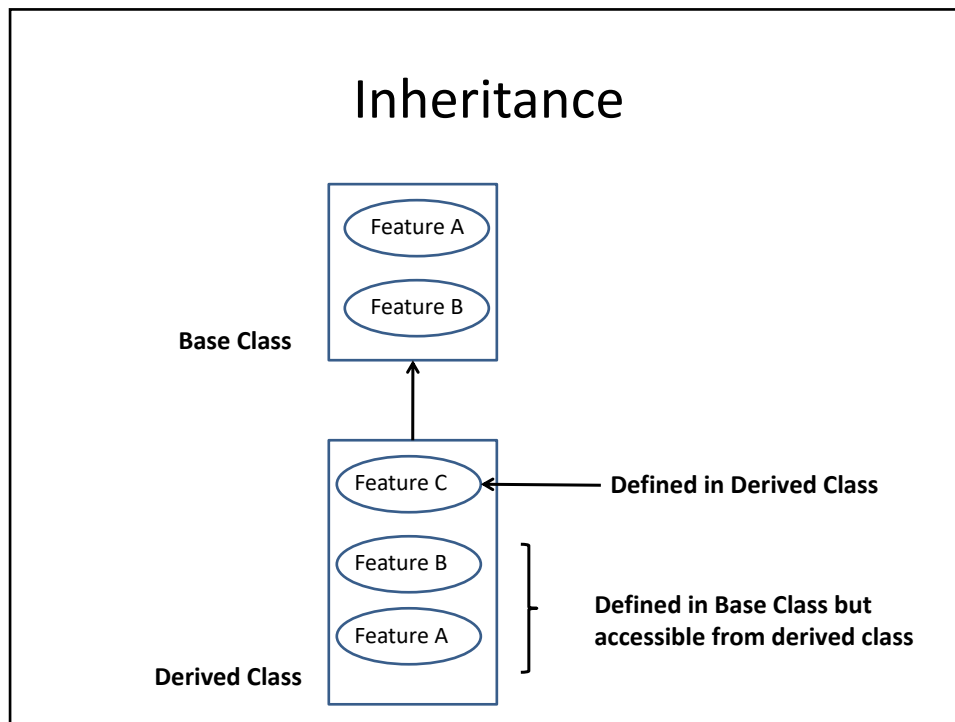


5

Inheritance Examples

Base Class	Derived Classes
Student	PG_Student, UG_Student
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
Account	CurrentAccount, SavingsAccount

6



7

Practical Meaning of Inheritance

- Data members in the parent are part of the child
- Behavior defined in the parent are part of the child
- Note that **private** aspects of the parent are *part* of the child, but are not (directly) accessible within the child class

Adapted From: *An Introduction to Object Oriented Programming*, 3rd Edition, by Timothy Budd

8

Inheritance

- Base and Derived classes.
- A derived class inherits the members of its base class.
- Friends are not inherited
- Syntax

`class` `derived-class:` `access-`
`specifier` `base-class`

where

Access-specifier is one of public, private or protected.

9

```
#include <iostream>

using namespace std;

// Base class
class Shape
{
    public:
        void setWidth(int w)
        {
            width = w;
        }
        void setHeight(int h)
        {
            height = h;
        }
    protected:
        int width;
        int height;
};
```

10

```
// Derived class
class Rectangle: public Shape
{
    public:
        int getArea()
        {
            return (width * height);
        }
};

int main(void)
{
    Rectangle Rect;
    Rect.setWidth(5);
    Rect.setHeight(7);

    // Print the area of the object.
    cout << "Total area: " << Rect.getArea() << endl;

    return 0;
}
```

11

OutPut

- Total area: 35

12

Kinds of Inheritance

- **Public** (Commonly used)
 - Public and protected members of the base class remain, respectively, public and protected members of the derived class.
- **Protected**
 - Public and protected members (described later) of the base class are inherited as protected members of the derived class.
- **Private**
 - Public and protected members of the base class are inherited as private members of the derived class.

In all cases, private members of a base class remain private to the base class and cannot be accessed **directly** by a derived class.

13

Access Control and Inheritance

- A derived class can access all the non-private members of its base class.
- Thus base-class members that should not be accessible to the member functions of derived classes should be declared private in the base class

14

Base class member access specifier	Kind of Inheritance		
	public inheritance	protected inheritance	private inheritance
public	public in derived class Can be accessed directly by member functions, friend functions, and non-member functions	protected in derived class Can be accessed directly by member functions and friend functions	private in derived class Can be accessed directly by member functions and friend functions
protected	protected in derived class Can be accessed directly by member functions and friend functions	protected in derived class Can be accessed directly by member functions and friend functions	private in derived class Can be accessed directly by member functions and friend functions
private	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class	Hidden in derived class Can be accessed by member functions and friend functions through public or protected member functions of the base class

15

protected Access Specifier

- **Note:** Not to be confused with protected Inheritance
- A base class's protected members can be accessed within that base class, by friends and members of that class, and by members and friends of any classes derived from that base class.
- Offers an intermediate level of protection between public and private access

16

Access Specifier	Accessible from Own class?	Accessible from Derived class?	Accessible from objects outside the class?
public	yes	yes	yes
protected	yes	yes	no
private	yes	No (not directly)	no

17

```

class counter
{
    protected:
        int count;
    public:
        counter(): count(0) { }
        counter (int c): count(c){ }
        int getcount() const
        { return count; }
        counter operator ++ () //prefix
        {
            return counter (++count);
        }
};

class countDn : public counter
{
    public:
        counter operator --() //prefix
        { return counter (--count); }
};

void main()
{
    countDn c1;
    cout<<c1.getcount(); //0
    ++c1;
    cout<<c1.getcount();
    --c1; cout<<c1.getcount();
}

```

18

In the previous example we cannot write:

```
counter c2;
```

```
c2= --c1;
```

Why?

19

Answer:

You cannot assign a base class object to a derived class object

(Reason)

Unclean behavior w.r.t. added members in the derived class

Note:

... but you can assign a derived class object to a base class object (because a derived class object **IS A base class object)**

20

INITIALIZING DERIVED CLASS OBJECTS THROUGH DERIVED CLASS CONSTRUCTORS

```
class counter
```

```
{
protected:
    int count;
public:
    counter(): count (0) {}
    counter(int c): count(c){}
    int getcount() const
    { return count; }
    counter operator ++()
    {return counter (++count);}
};
```

```
class countDn: public counter
```

```
{ public:
    countDn():counter() { }
    countDn(int c): counter (c){ }
    countDn operator -- ()
    { return countDn(--count); }
};

void main()
{
    countDn c1, c2(100), c3;
    cout<<c1.getcount(); //0
    ++c1; cout<<c1.getcount();//1
    c3= --c2; cout<<c3.getcount(); }
```

21

Multiple Inheritance

- A C++ class can inherit members from more than one class
- `class derived-class: access baseA, access baseB...`

22

```

#include <iostream>

using namespace std;

// Base class Shape
class Shape
{
public:
    void setWidth(int w)
    { width = w;}
    void setHeight(int h)
    { height = h; }
protected:
    int width;
    int height;
};

// Base class PaintCost
class PaintCost
{
public:
    int getCost(int area)
    {
        return area * 70;
    }
};

// Derived class
class Rectangle: public Shape,
public PaintCost
{
public:
    int getArea()
    {
        return (width *
height);
    }
};

```

23

```

int main(void)
{
    Rectangle Rect;
    int area;

    Rect.setWidth(5);
    Rect.setHeight(7);

    area = Rect.getArea();

    // Print the area of the object.
    cout << "Total area: " << Rect.getArea() << endl;

    // Print the total cost of painting
    cout << "Total paint cost: $" << Rect.getCost(area) << endl;

    return 0;
}

```

```

Total area: 35
Total paint cost: $2450

```

24

Overriding Member Functions

- Overriding refers to **re-defining the base class member functions within the derived classes**.
- When a derived class has an over-riding version of a base class function, the **base class version** can still be accessed (by a derived class object) using the binary scope resolution operator i.e. **::**

25

EXAMPLE: OVERRIDING MEMBER FUNCTIONS

```
#include <cstdlib>
class stack {
    protected:
        int a[10];
        int top;
    public:
        stack() { top=-1; }
        void push(int var)
        { a[++top]=var; }
        int pop()
        { return a[top--]; }
};
```

```
class stack2 : public stack {
    public:
        void push(int var) {
            if (top>=9)
                { cout<<"Error: stack full"; exit(1); }
            stack::push(var);
        }
        int pop() {
            if (top < 0)
                { cout<<"Error: stack empty"; exit(1); }
            return stack::pop ();
        }
    void main()
    { stack2 s;   s.push(15); s.push(10);
      cout<<s.pop(); }
```

26

The Effects of Inheritance

- Recall that the constructors of member objects (in composition) are executed before the constructor of the class containing them. The same can be said about the constructor of the base class
- Also recall that the destructors of member objects are executed after the destructor of the class containing them. This is true for the destructor of the base class as well

27

Order of Execution of Constructors in Inheritance

- When a program creates a derived-class object, the derived-class constructor immediately calls the base-class constructor; the **base-class constructor's body** executes, then the **derived-class's member initializers** execute and finally the **derived-class constructor's body** executes.
- This process **cascades up** the hierarchy if it contains more than two levels.

28

Order of Execution of Destructors in Inheritance

- When a derived-class object is destroyed, the program calls that object's destructor. This begins a chain(cascade) of destructor calls in which the derived-class destructor and the destructors execute in **reverse of the order in which the constructors executed**.

29

Example: Order of Execution of Ctor/Dtor

- Suppose:


```
class lightSource { ... };
class bulb: public lightSource {
    string manufacturer;
    dateSpan expectedLife;
    ...};
```
- The **order of constructor execution** is
 - lightSource
 - string (for manufacturer)
 - dateSpan (for expectedLife)
 - bulb
- The **order of destructor execution** is
 - bulb
 - dateSpan (for expectedLife)
 - string (for manufacturer)
 - lightSource

30

Class Work

- Create an inheritance hierarchy that a bank might use to represent customers' **bank accounts**. All customers at this bank can **deposit** (i.e. credit) money into their accounts and **withdraw** (i.e. debit) money from their accounts. More specific types of accounts also exist. **Savings accounts**, for instance, earn interest on the money they hold. **Checking accounts**, on the other hand, charge a fee per transaction (i.e. credit or debit).

31

Class Work

- Create an inheritance hierarchy containing base class Account and derived classes SavingsAccount that inherit from class Account.
- Base class (Account) members:
 - Data: double, balance
 - Functions:
 - Constructor (argument: initial balance)
 - Credit: add an amount to current balance
 - Debit: withdraw money from account
 - getBalance: return current value of balance

32

Class Work

- Derived class SavingsAccount
- Added data member: double, interest rate
- Added member functions:
 - Constructor: (argument 1: initial balance, argument 2: interest rate)
 - calculateInterest: returns a double indicating the amount of interest earned by an account. This amount should be determined by multiplying interest rate by account balance.

33

Class Work

- Derived class CheckingAccount
- Added data member: double, fee charged per transaction
- Added member functions:
 - Constructor: (argument 1: initial balance, argument 2: fee amount)
 - Redefined member functions credit and debit:
 - subtract fee from account balance whenever either transaction is performed successfully
 - These functions should invoke the base class versions to update the account balance

34

Class Work

- Write a program that creates objects of each class and tests their member functions.

35

Benefits of Inheritance

- Software Reuse
- Code Sharing
- Improved Reliability
- Consistency of Interface
- Rapid Prototyping
- Information Hiding

Adapted From: *An Introduction to Object Oriented Programming*, 3rd Edition, by Timothy Budd

36

Cost of Inheritance

- Execution speed
 - Program size
 - Message Passing Overhead
 - Program Complexity
-
- This does not mean you should not use inheritance, but rather than you must understand the benefits, and weigh the benefits against the costs

Adapted From: *An Introduction to Object Oriented Programming*, 3rd Edition, by Timothy Budd